

QuickFIX/J 自学纲领：从业务流程到会话可靠性

目标：用一笔订单说明业务如何接入，用消息转换说明 FIX 协议如何落地，用会话建立、动态会话建立及会话恢复说明生产环境如何保证通信可靠。

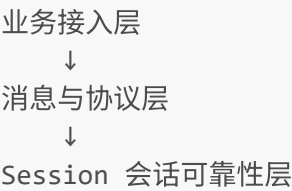
适用场景：学习 QuickFIX/J、向团队介绍 FIX 接入架构、准备 30~45 分钟的技术分享。

1. 核心结论

用三个方向介绍 QuickFIX/J 的总体思路是正确的：

- 1. NewOrderSingle 的完整下单流程
- 2. 领域 Order、FIX Message 与原始 FIX 报文之间的转换
- 3. 断线、重连、序号恢复与消息重传

它们对应 QuickFIX/J 的三层核心能力：

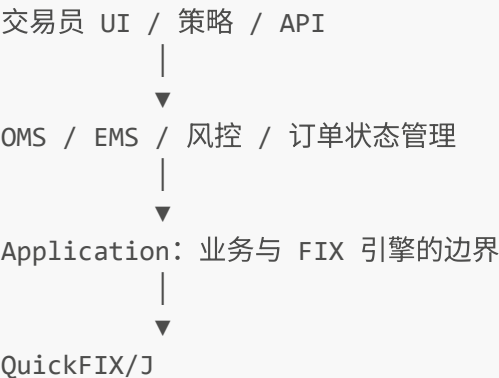


但是，在进入这三条主线前，需要先说明 **QuickFIX/J 的定位与边界**；第三条也不应只称为“断线重连”或“高可用”，而应提升为：

Session 可靠性：心跳、断线、重连、序号恢复与消息重传

2. 第零部分：QuickFIX/J 的定位与边界

2.1 QuickFIX/J 所在的位置



└ Session: 会话、序号、重连、心跳、重传
└ Message: FIX 消息对象与字段模型
└ DataDictionary: 协议合法性校验规则
└ MessageStore: 消息与序号持久化
└ MINA / TCP: 网络通信
 ↓
券商 / 交易所 / ECN / 内部撮合服务

2.2 必须明确的边界

QuickFIX/J 是 Java 的 FIX 协议与会话引擎。

QuickFIX/J 不是：

- 完整 OMS
- 完整 EMS
- 风控系统
- 撮合引擎
- 行情系统
- 系统级高可用平台

以示例为例：

Banzai

= 带 UI 的示例买方交易终端

OrderMatch

= 带内存订单簿的示例撮合服务

Executor

= 多版本订单执行服务示例

QuickFIX/J

= 支撑上述示例进行 FIX 消息收发、解析、校验和会话管理的引擎

3. 主线一：一笔 NewOrderSingle 的端到端业务流程

3.1 这一部分要回答的问题

- 业务代码如何使用 QuickFIX/J？
- 一笔订单从哪里产生、如何被发送、如何被接收？
- ExecutionReport 如何回到买方系统？
- QuickFIX/J 与业务实现的边界在哪里？

3.2 完整业务链路

Banzai (买方 / Initiator)

OrderEntryPanel

| 用户填写 Symbol、Quantity、Side、Type 等



Banzai Order

| 本地领域订单模型



BanzaiApplication.send(order)

| 根据 SessionID 的 BeginString 选择 FIX 版本



send42(order)



NewOrderSingle (35=D)



Session.sendToTarget(...)



QuickFIX/J Session

| 补充 Header

| 分配 MsgSeqNum(34)

| 持久化消息与序号

| 计算 BodyLength(9)

| 计算 CheckSum(10)



TCP / Apache MINA



OrderMatch (卖方 / Acceptor)

TCP / Apache MINA



Session.next(...)



Application.fromApp(message, sessionID)



MessageCracker.crack(message, sessionID)



onMessage(NewOrderSingle, sessionID)



OrderMatch Order

| 卖方 / 撮合服务自己的领域订单模型

```

▼
OrderMatcher / Market
|
▼
ExecutionReport (35=8)
|
▼
Session.sendToTarget(...)
|
▼
BanzaiApplication.fromApp(...)
|
▼
订单状态、成交数量、均价等更新到 Banzai

```

3.3 三层职责划分

```

业务层
├─ Banzai Order
├─ UI、OMS、风控、策略
├─ OrderMatch 的撮合逻辑
└─ 订单状态机

QuickFIX/J 引擎层
├─ Application
├─ Session
├─ MessageCracker
├─ MessageStore
├─ SocketInitiator
└─ SocketAcceptor

FIX 协议层
├─ NewOrderSingle (35=D)
├─ ExecutionReport (35=8)
├─ Header / Body / Trailer
├─ Tag=Value
└─ DataDictionary

```

3.4 订单标识与订单状态管理

订单链路中不能只关注“订单是否发送成功”，还必须明确不同订单号的来源、生命周期和关联关系。以 **Banzai** -> **Executor** 为例，至少要区分以下 4 个标识：

```

Banzai Order.ID
= 买方本地领域订单 ID

ClOrdID(11)
= 买方发送给执行端的客户订单 ID

```

OrderID(37)
= Executor / 卖方为订单分配的执行端订单 ID

ExecID(17)
= Executor 为每一次执行或回报事件分配的 ID

ID 的生成和传递链

用户点击 Submit

- > new Banzai Order()
- > Order.generateID()
- > 得到本地 Order.ID
- > 构造 NewOrderSingle
- > ClOrdID(11) = Order.ID
- > 发送给 Executor
- > Executor 创建 ExecutionReport
- > genOrderID() 生成 OrderID(37)
- > genExecID() 生成 ExecID(17)
- > 回报中同时携带 ClOrdID / OrderID / ExecID



为什么需要多个 ID

ClOrdID(11)
用于买方把回报关联回自己的本地订单。

OrderID(37)
用于执行端在自己的订单系统中识别这笔订单。

ExecID(17)
用于区分同一订单生命周期中的不同回报事件。

Banzai 接收 `ExecutionReport` 后，通常通过 `ClOrdID(11)` 找回本地订单：

```
ExecutionReport.ClOrdID(11)
-> Banzai Order.ID
-> 更新订单状态、成交数量、均价和 UI
```

订单状态可以抽象为：

```
本地订单创建
-> NewOrderSingle 已发送
-> ExecutionReport: New
-> Partially Filled
-> Filled / Canceled / Rejected
```

撤单时还要区分原订单和撤单请求：

```
原始订单: ClOrdID(11)=C001
撤单请求: OrigClOrdID(41)=C001, ClOrdID(11)=C002
```

当前 Executor 示例与生产系统的差异

当前示例在每次构造 `ExecutionReport` 时调用 `genOrderID()`，因此同一订单的 `NEW` 和 `FILLED` 回报可能得到不同的 `OrderID(37)`。这是为了演示回报构造的简化实现。

生产系统通常应保持：

```
同一订单生命周期: OrderID(37) 保持不变
不同回报事件: ExecID(17) 分别递增或唯一

New                OrderID=0001, ExecID=E001
PartiallyFilled    OrderID=0001, ExecID=E002
Filled             OrderID=0001, ExecID=E003
```

因此，订单状态管理的核心不是简单生成一个数字，而是建立稳定的关联关系：

```
ClOrdID  -> 买方订单
OrderID  -> 执行端订单
ExecID   -> 执行事件
```

相关示例源码

```
Banzai:
- banzai/Order.java: new Order() 时生成本地 Order.ID
```

- banzai/BanzaiApplication.java: 将 Order.ID 写入 ClOrdID(11)
 - banzai/BanzaiApplication.java: 通过 ClOrdID 找回本地订单
- Executor:
- executor/Application.java: genOrderID() 生成 OrderID(37)
 - executor/Application.java: genExecID() 生成 ExecID(17)
 - executor/Application.java: 创建并发送 ExecutionReport

3.5 Banzai 到 Executor：一笔订单与 ExecutionReport 的完整链路

本节以默认多版本配置中的 FIX 4.2 会话为例：

Banzai 本地 Session: FIX.4.2:BANZAI->EXEC
Executor 本地 Session: FIX.4.2:EXEC->BANZAI
默认监听端口：9878

Executor 不维护像 OrderMatch 那样的订单簿撮合；它校验订单后先回一条“已接收”回报，若订单满足其模拟执行条件，再回一条“已成交”回报。

3.5.1 Banzai → Executor → Banzai 的通信流程





└─ avgPx = AvgPx(6)

└─ 增加 Execution: symbol、quantity、lastPx、side

└─ 更新 OrderTableModel / ExecutionTableModel / UI

3.5.2 标准语义下，本地订单状态如何变化

以买入 AAPL 100 @ 123.45，且 Executor 决定立即完全成交为例：

阶段	对应 FIX 消息与关键字段	Banzai 本地 Order 的目标状态
创建	本地 Order	open=100、executed=0、avgPx=0、isNew=true
已接收	35=8、150=0(New)、39=0(New)、14=0、151=100	isNew=false；未成交，因此open=100、executed=0
完全成交	35=8、150=2(Fill)、39=2(Filled)、32=100、31=123.45、14=100、6=123.45、151=0	open=0、executed=100、avgPx=123.45；新增一条成交记录

部分成交时，状态应按每条执行回报逐步推进：



每次新的执行事件都必须使用新的 ExecID(17)。Banzai 用 SessionID + ExecID 记忆已经处理的回报；这是防止 Session 重传或重复投递造成重复记账、重复更新订单状态的重要业务级幂等保护。

3.5.3 当前示例代码的字段一致性问题

当前 Executor 的 FIX 4.2 New 回报在 executor/Application.java:274-279 中构造为：

OrdStatus=New

CumQty=0

LeavesQty=0

但按标准订单语义，尚未成交的新订单应满足：

CumQty=0

LeavesQty=OrderQty

这不是一个无关紧要的展示细节。Banzai 的 `executionReport(...)` 在缺少 `LastShares(32)` 时，会按下面的公式推导本次成交量：

```
fillSize = OrderQty - LeavesQty
```

所以，按**当前示例的实际代码**，若 Banzai 收到上述 `LeavesQty=0` 的 `New` 回报，会错误地推导出整笔订单已经成交，导致本地 `open` 被提前减为 0；随后再收到 `Filled` 回报，可能再次扣减成交量。这是 Banzai 与 Executor 示例组合中的订单状态字段不一致问题。

学习时应分别记住：

消息链路是正确的：

`NewOrderSingle` -> `New ExecutionReport` -> `Filled ExecutionReport`。

字段状态模型需要修正后才可作为生产参考：

`New` 时 `LeavesQty`=订单剩余量；

`Partial` / `Fill` 时 `CumQty`、`LeavesQty`、`LastQty` / `LastShares`、`AvgPx`、`OrdStatus` 与 `ExecType` 必须彼此一致。

3.5.4 关键源码职责

Banzai 发单

```
BanzaiApplication.send(...)
-> send42(order)
-> quickfix.fix42.NewOrderSingle
-> Session.sendToTarget(...)
```

Executor 接单与回报

```
executor/Application.onMessage(fix42.NewOrderSingle, sessionID)
-> validateOrder(...)
-> 创建 New 的 ExecutionReport
-> sendMessage(sessionID, accept)
-> isOrderExecutable(...)
-> 若可执行，创建 Filled 的 ExecutionReport
-> sendMessage(sessionID, executionReport)
```

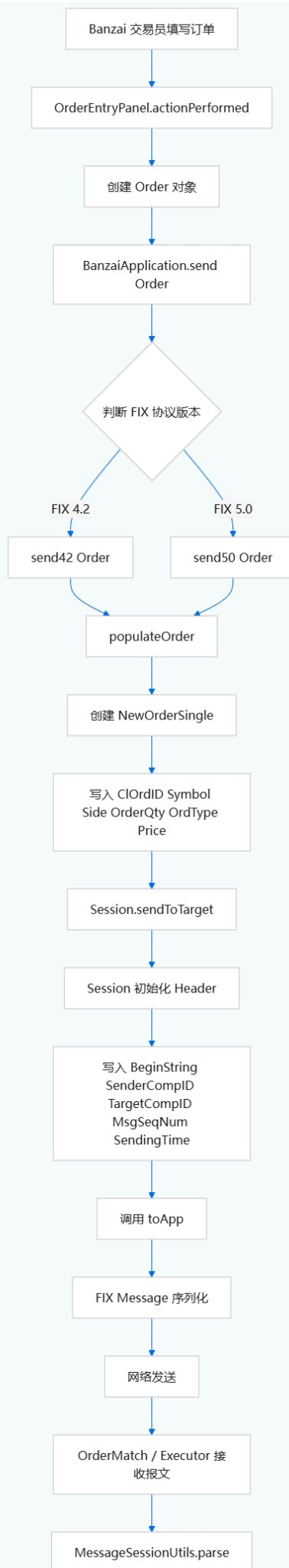
Banzai 更新订单与成交

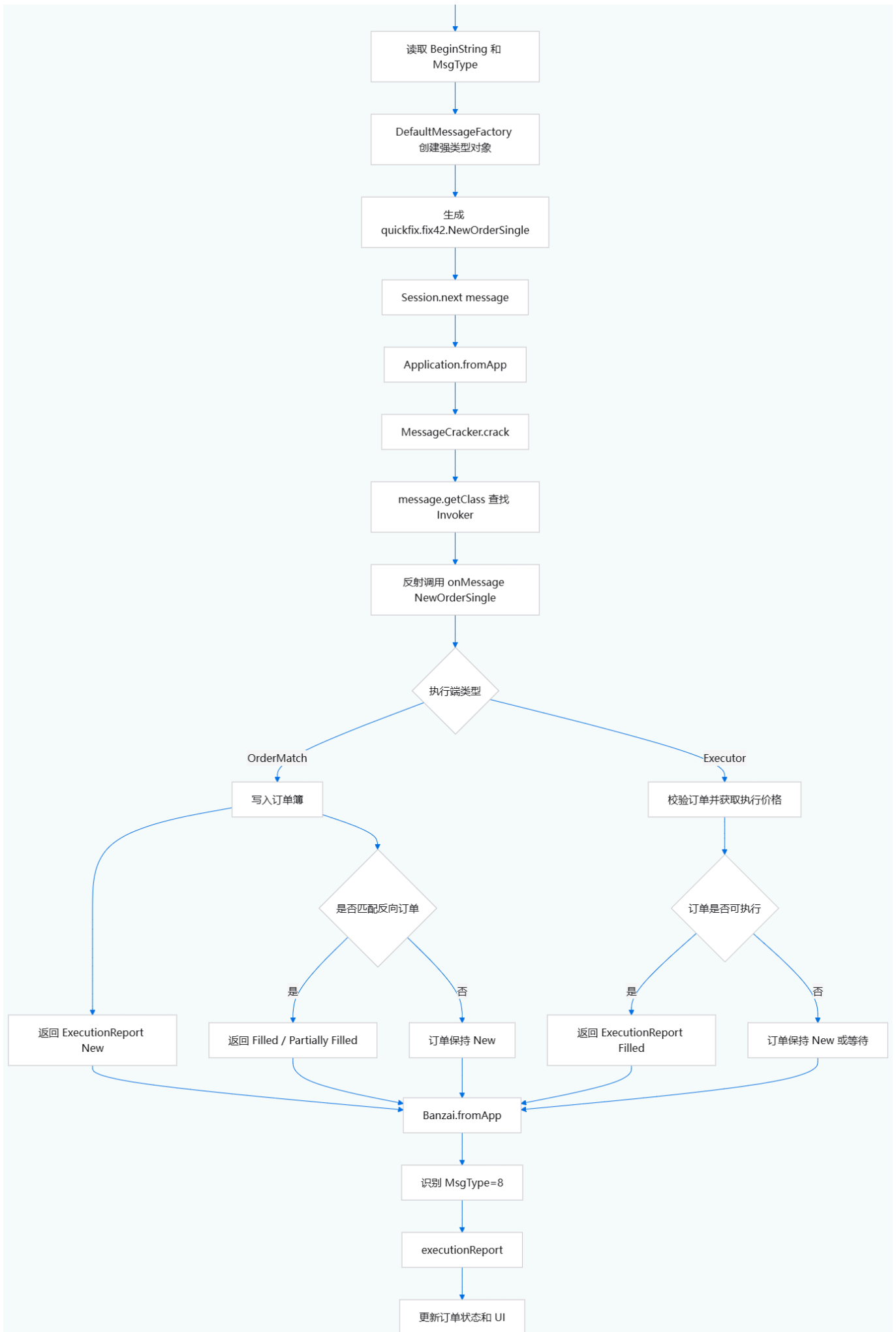
```
BanzaiApplication.fromApp(...)
-> Swing EDT 的 MessageProcessor.run()
-> executionReport(message, sessionID)
-> OrderTableModel.updateOrder(...)
-> ExecutionTableModel.addExecution(...)
```

3.6 对外介绍时的关键结论

一笔订单的业务含义由 OMS、EMS、风控、撮合等业务系统决定；QuickFIX/J 负责把该业务意图可靠地表达为 FIX 消息，并在对手方之间安全、正确地传递。订单状态管理还必须明确 `ClOrdID`、`OrderID` 和 `ExecID` 的生成方、关联方式与生命周期。

3.7 图片总结





4. 主线二：领域对象、FIX Message 与原始 FIX 报文的转换

4.1 更严谨的表达

不要笼统地说：

Order 变成 FIX 协议，再变成 Order。

建议使用下面的表达：

买方领域 Order

- FIX 强类型消息 NewOrderSingle
- 原始 FIX Tag=Value 报文
- 接收方强类型消息 NewOrderSingle
- 卖方领域 Order

4.2 对象与报文的完整转换链路

Banzai / 买方

领域对象

`quickfix.examples.banzai.Order`

|
| `BanzaiApplication.send42(order)`



协议对象

`quickfix.fix42.NewOrderSingle`

|
| Session 编码并添加会话字段



原始 FIX 报文

`8=FIX.4.2|9=...|35=D|49=BANZAI|56=EXEC|34=2|`
`11=ORD-001|55=AAPL|54=1|38=100|40=2|44=123.45|10=...|`



TCP 字节流

OrderMatch / 卖方

TCP 字节流



原始 FIX 报文

|
| 根据 `BeginString(8)` 与 `MsgType(35)` 选择消息类型



协议对象

`quickfix.fix42.NewOrderSingle`

```

|
| Application.onMessage(...)
▼
领域对象
quickfix.examples.ordermatch.Order
|
▼
撮合 / 执行 / 风控等卖方业务逻辑

```

上图中的 | 仅用于展示；实际 FIX 字段分隔符是 SOH 字符 (`\001`)。

4.3 三种“订单”不是同一个对象

```

Banzai Order
└─ Banzai 示例的本地业务对象，服务于 UI 和订单状态维护。

NewOrderSingle
└─ FIX 规范定义的协议消息，MsgType(35)=D。

OrderMatch Order
└─ OrderMatch 示例的内部领域对象，服务于订单簿与撮合。

```

因此，QuickFIX/J 不会自动把任意业务类映射为 FIX 消息。正确理解是：

```

业务代码负责：
领域对象 ↔ FIX Message 的映射。

QuickFIX/J 负责：
FIX Message ↔ 原始 FIX 报文 ↔ 网络传输的处理。

```

4.4 QuickFIX/J 消息模型的核心对象

```

FieldMap
└─ 保存 Tag -> Value 字段
└─ 保存重复组 (Group)

Message
└─ Header
└─ Body
└─ Trailer

DataDictionary
└─ 字段类型
└─ 必填字段
└─ 枚举值
└─ 消息定义
└─ 重复组定义

```


DefaultMessageFactory

- └ 根据 BeginString(8) 与 MsgType(35) 创建正确的强类型消息。

4.5 Header、Body、Trailer

Header：会话和路由信息

```
8=FIX.4.2      BeginString
35=D           MsgType: NewOrderSingle
49=BANZAI      SenderCompID
56=EXEC        TargetCompID
34=2           MsgSeqNum
```

Body：业务字段

```
11=ORD-001     ClOrdID
55=AAPL        Symbol
54=1           Side (Buy)
38=100         OrderQty
40=2           OrdType (Limit)
44=123.45      Price
```

Trailer：完整性校验

```
10=...         CheckSum
```

4.6 FieldMap 中的 Group：用行情快照理解重复组

FIX 消息除了普通的 **Tag -> Value** 字段，还需要表达“同一结构重复出现”的数据。QuickFIX/J 用 **FieldMap** 的 **Group** 表示这种 FIX Repeating Group。

最直观的例子是行情快照：一个 Symbol 同时包含 Bid、Ask，甚至多个深度档位。

MarketDataSnapshotFullRefresh

```
├ Symbol(55) = AAPL
└ NoMDEntries(268) = 2
  ├── Group 1
  │   ├── MDEntryType(269) = Bid
  │   ├── MDEntryPx(270) = 100.10
  │   └── MDEntrySize(271) = 1000
  └── Group 2
      ├── MDEntryType(269) = Offer / Ask
      ├── MDEntryPx(270) = 100.12
      └── MDEntrySize(271) = 1200
```

对应的 FIX 业务字段可以简化为：

```

55=AAPL
268=2
269=0 | 270=100.10 | 271=1000
269=1 | 270=100.12 | 271=1200

```

其中：

```

268=2
= 后面有 2 条重复行情记录

每一个 Group
= 一条完整的 Bid / Ask / 深度行情记录

```

4.6 普通字段与 Group 的内部结构

```

普通字段：
FieldMap.fields
└─ Tag -> Value
    55 -> AAPL

重复组：
FieldMap.groups
└─ Group 标识字段 -> List<Group>
    268 -> [
        {269 -> 0, 270 -> 100.10, 271 -> 1000},
        {269 -> 1, 270 -> 100.12, 271 -> 1200}
    ]

```

因此：

```

FieldMap
├─ fields: 保存普通字段
└─ groups: 保存重复组列表

```

Message 负责把这些内容组织到完整消息中：

```

Message
├─ Header: 会话字段
├─ Body: Symbol、NoMDEntries 和 Group 列表
└─ Trailer: 校验字段

```

4.7 Group 的构造与读取

构造行情消息时，业务代码不是手工拼接所有 `Tag=Value`，而是创建强类型 `Group` 并加入消息：

```
MarketDataSnapshotFullRefresh.NoMDEntries bid =
    new MarketDataSnapshotFullRefresh.NoMDEntries();
bid.set(new MDEntryType(MDEntryType.BID));
bid.set(new MDEntryPx(100.10));
bid.set(new MDEntrySize(1000));
message.addGroup(bid);

MarketDataSnapshotFullRefresh.NoMDEntries ask =
    new MarketDataSnapshotFullRefresh.NoMDEntries();
ask.set(new MDEntryType(MDEntryType.OFFER));
ask.set(new MDEntryPx(100.12));
ask.set(new MDEntrySize(1200));
message.addGroup(ask);
```

接收方按组的序号读取：

```
MarketDataSnapshotFullRefresh.NoMDEntries group =
    new MarketDataSnapshotFullRefresh.NoMDEntries();

message.getGroup(1, group); // Bid
double bidPx = group.getDouble(MDEntryPx.FIELD);

message.getGroup(2, group); // Ask
double askPx = group.getDouble(MDEntryPx.FIELD);
```

核心流程是：

```
addGroup(...)
-> FieldMap 保存 Group 列表
-> Session 序列化为重复 FIX 字段
-> 对端解析 NoMDEntries
-> getGroup(index, group)
-> 读取每条行情记录
```

4.8 DataDictionary 与 Group 的关系

`FieldMap` 负责“存储 Group”，但不负责定义 Group 的协议规则。Group 的定义来自 `DataDictionary`：

```
DataDictionary
├─ 268 是 NoMDEntries
├─ 269 是 Group 的起始字段 / delimiter
├─ 270、271 属于该 Group
```

- └ 每组允许哪些字段
- └ Group 的顺序、类型和数量规则

解析过程可以理解为：

```

读到 268=2
  -> 知道后面有 2 组
读到第一个 269
  -> 开始 Group 1
读到下一个 269
  -> Group 1 结束，开始 Group 2
读完 Group 2
  -> 重复组结束
  
```

所以三者关系是：

```

DataDictionary
= 定义 Group 应该怎样解析、怎样校验

Message
= 组织包含 Group 的完整 FIX 消息

FieldMap
= 保存普通字段和 Group 列表
  
```

4.9 行情请求中的 Group

Group 不只出现在行情快照中，行情订阅请求也经常使用 Group：

```

MarketDataRequest
└ NoMDEntryTypes(267)
  └ Group 1: Bid
  └ Group 2: Offer
└ NoRelatedSym(146)
  └ Group 1: AAPL
  └ Group 2: MSFT
  └ Group 3: TSLA
  
```

这表达的是：

```

一次请求订阅多个行情类型
一次请求订阅多个 Symbol
  
```

4.10 Group 的实战意义

在行情系统中：

一个 Group = 一条 Bid / Ask / 深度行情
多个 Group = 一个 Symbol 的行情集合或订单簿档位

例如：

```
AAPL
├─ Bid Level 1
├─ Bid Level 2
├─ Ask Level 1
└─ Ask Level 2
```

因此 Group 不只是“重复字段的语法”，而是表达以下结构化业务数据的基础：

- 行情档位
- 多个订阅 Symbol
- 多种行情类型
- 订单分配明细
- 执行明细
- 账户或持仓明细

理解 Group，是从“会读取普通 FIX 字段”进入“能处理真实复杂 FIX 消息”的关键一步。

4.11 新增自定义字段：以 `InvestmentDecisionID(9001)` 为例

当标准 FIX 字段不足以表达双方的业务需求时，可以在双方协商的 **Rules of Engagement（对接规则）** 中增加用户自定义字段。以下示例在订单中携带买方的“投资决策编号”，用于关联策略信号、委托、执行回报与审计记录：

字段名：InvestmentDecisionID
Tag：9001（用户自定义字段通常使用 ≥ 5000 的 Tag）
类型：STRING

发送：NewOrderSingle(35=D)
回传：ExecutionReport(35=8)
示例值：ALPHA-AAPL-20260823-001

4.11.1 最小实现的文件清单

新增文件：2 个

```
├─ quickfixj-examples/banzai/src/main/resources/
├─ quickfixj-examples/banzai/FIX42_BUYSIDE.xml
├─ quickfixj-examples/executor/src/main/resources/
└─ quickfixj-examples/executor/FIX42_BUYSIDE.xml
```

修改文件：4 个

```
├─ banzai/BanzaiApplication.java
├─ executor/Application.java
├─ Banzai 使用的 cfg
└─ Executor 使用的 cfg
```

两份 `FIX42_BUYSIDE.xml` 应从标准 `FIX42.xml` 复制而来，并保持内容一致。不要长期直接修改仓库内的标准字典。

4.11.2 在自定义字典中增加什么

第一处：在 `<fields>` 中定义 Tag、名称和类型：

```
<field number="9001" name="InvestmentDecisionID" type="STRING"/>
```

第二处：在 `NewOrderSingle(35=D)` 中声明该字段可用：

```
<message name="NewOrderSingle" msgtype="D" msgcat="app">
  <!-- 原有标准字段保持不变 -->
  <field name="InvestmentDecisionID" required="N"/>
</message>
```

第三处：在 `ExecutionReport(35=8)` 中声明可回传该字段：

```
<message name="ExecutionReport" msgtype="8" msgcat="app">
  <!-- 原有标准字段保持不变 -->
  <field name="InvestmentDecisionID" required="N"/>
</message>
```

`<fields>` 中的定义：说明“9001 是什么、值是什么类型”。

`<message>` 中的引用：说明“9001 可以出现在哪类 FIX 消息中”。

这里使用 `required="N"`，表示对接初期允许旧订单或手工订单不带该字段。只有当双方明确所有相关消息都必须携带该字段时，才应改为 `required="Y"`。

4.11.3 在两端配置中加载相同字典

在 Banzai 与 Executor 各自实际使用的 Session 配置中，将：

```
DataDictionary=FIX42.xml
```

改为：

```
DataDictionary=FIX42_BUYSIDE.xml
```

Banzai 认识 9001、Executor 不认识 9001
-> Executor 可能因字典校验拒绝订单。

Executor 回传 9001、Banzai 不认识 9001
-> Banzai 可能因字典校验拒绝回报。

因此，自定义字段的字典必须作为双方共同版本化的协议契约。

4.11.4 Banzai 需要增加的发送与接收逻辑

修改：

```
quickfixj-examples/banzai/src/main/java/  
quickfix/examples/banzai/BanzaiApplication.java
```

在 `send42(Order order)` 创建 `NewOrderSingle` 并设置 `OrderQty` 后，增加：

```
newOrderSingle.setString(9001, "LEARNING-ALPHA-001");
```

这会使 `NewOrderSingle(35=D)` 的 Body 中包含：

```
9001=LEARNING-ALPHA-001
```

在 `executionReport(Message message, SessionID sessionID)` 开头增加读取逻辑：

```
if (message.isSetField(9001)) {  
    System.out.println("ExecutionReport InvestmentDecisionID="  
        + message.getString(9001)  
        + ", session=" + sessionID
```

```
        + ", clOrdID=" + message.getString(ClOrdID.FIELD));
    }
```

最小学习版先使用固定字符串即可；后续应在 Banzai 的领域 `Order` 中增加 `investmentDecisionID` 属性，再将：

```
Order.investmentDecisionID
-> NewOrderSingle Tag 9001
```

从而实现真正的领域对象到 FIX Message 映射。

4.11.5 Executor 需要增加的读取和回传逻辑

修改：

```
quickfixj-examples/executor/src/main/java/
quickfix/examples/executor/Application.java
```

在：

```
onMessage(quickfix.fix42.NewOrderSingle order, SessionID sessionID)
```

的 `validateOrder(order)` 后，读取字段：

```
String investmentDecisionID = null;
if (order.isSetField(9001)) {
    investmentDecisionID = order.getString(9001);
    System.out.println("Received InvestmentDecisionID="
        + investmentDecisionID + ", session=" + sessionID);
}
```

创建 `New` 的 `ExecutionReport` 后、发送前，回传字段：

```
if (investmentDecisionID != null) {
    accept.setString(9001, investmentDecisionID);
}
```

创建 `Filled` 的 `ExecutionReport` 后、发送前，同样回传：


```
if (investmentDecisionID != null) {
    executionReport.setString(9001, investmentDecisionID);
}
```

4.11.6 完整验证流程

```
Banzai
|
| NewOrderSingle(35=D)
| 11=C001
| 9001=LEARNING-ALPHA-001
▼
Executor
|
| DataDictionary 校验: 9001 已定义且允许出现在 35=D
| 读取并打印 9001
| 可将其用于风控、路由、审计或策略归因
|
| ExecutionReport(35=8, New)
| 9001=LEARNING-ALPHA-001
|
| ExecutionReport(35=8, Filled)
| 9001=LEARNING-ALPHA-001
▼
Banzai
|
| DataDictionary 校验: 9001 已定义且允许出现在 35=8
| 读取并打印 9001
| 将订单、成交和投资决策记录关联
```

4.11.7 后续规范化：新增类型安全字段类

最小实现使用：

```
message.setString(9001, value);
message.getString(9001);
```

后续可新增共享 Java 类：

```
quickfixj-examples/custom-fix-rules/src/main/java/
quickfix/examples/common/field/InvestmentDecisionID.java
```

```
package quickfix.examples.common.field;

import quickfix.StringField;

public class InvestmentDecisionID extends StringField {
    public static final int FIELD = 9001;

    public InvestmentDecisionID() {
        super(FIELD);
    }

    public InvestmentDecisionID(String value) {
        super(FIELD, value);
    }
}
```

Banzai 与 Executor 共用该类后，代码可改为：

```
newOrderSingle.setField(
    new InvestmentDecisionID("LEARNING-ALPHA-001"));
```

FIX42_BUYSIDE.xml
= 双方认可的协议定义与校验规则。

InvestmentDecisionID.java
= Java 侧更清晰、类型安全的字段访问工具。

4.11.8 不建议长期关闭校验

QuickFIX/J 提供：

```
ValidateUserDefinedFields=N
```

它允许 Tag >= 5000 的未知字段绕过部分字典校验，适合短期排障或字典迁移期；但不应作为正式对接方案。

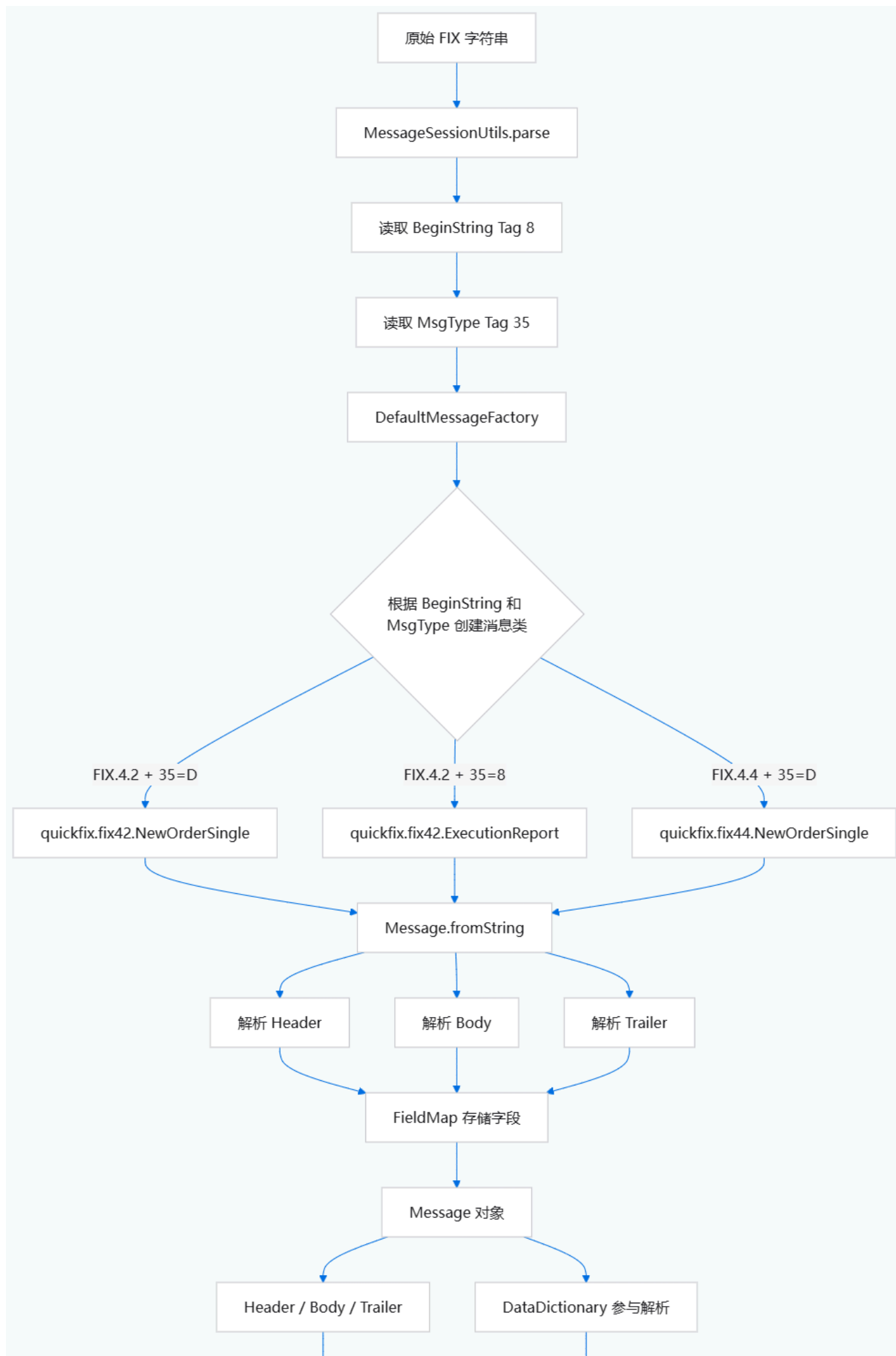
正式方案：
双方使用同版本自定义 DataDictionary
+
ValidateUserDefinedFields=Y

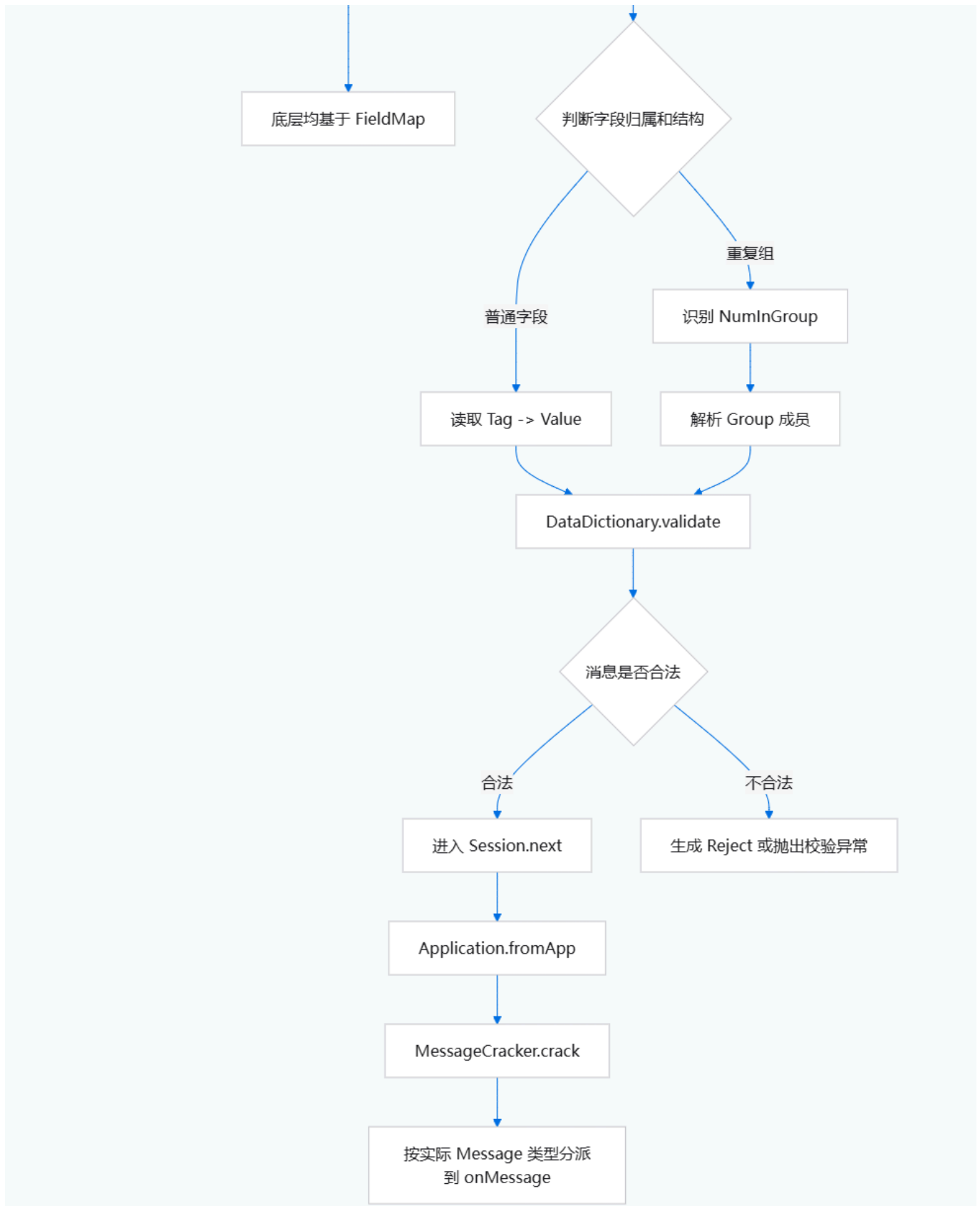
不推荐：
长期关闭校验，把自定义 Tag 当作无约束字符串。

新增自定义字段不是只写一行 `setString(9001, value)`；完整流程是：业务语义定义 → 双方约定 Tag 与类型 → `DataDictionary` 声明字段与消息归属 → 双方加载相同字典 → 发方设置 → 收方读取并回传 → 回报、审计和幂等处理关联。

4.12 对外介绍时的关键结论

`FieldMap.fields` 保存普通字段，`FieldMap.groups` 保存重复结构；`Message` 负责组织完整消息，`DataDictionary` 定义 Group 的起始字段、成员字段、顺序和校验规则。在行情场景中，一个 Group 表示一条 Bid、Ask 或深度行情记录，多个 Group 共同构成一个 Symbol 的行情集合。自定义字段则进一步表明：`DataDictionary` 不只描述标准 FIX，它也是双方共同维护的业务协议契约。





5. 主线三：Session 可靠性——心跳、重连、序号与重传

Session.java 的整体架构：四条主线

`Session.java` 不是简单的 TCP 连接类，而是 QuickFIX/J 中围绕 FIX 会话生命周期运行的核心状态机。它把网络连接、FIX 协议校验、消息分发、序号管理、心跳和断线恢复组织在一起。



```
Session
= FIX 逻辑会话
≠ 一次 TCP 连接
```

同一个 **SessionID** 可以经历多次 TCP 连接；连接断开时，当前 **Responder** 会被清理，但 Session 的序号、Store 和恢复状态可以继续保留。

1. 入站消息处理线

```
TCP / Apache MINA
-> Connector 将字节解析为 Message
-> Session.next(Message)
-> 检查 MsgSeqNum、BeginString、CompID、时间和字典
-> verify(...)
-> 按 MsgType 分流
```

管理消息由 **Session** 自己处理：

```
Logon           -> nextLogon(...)
Heartbeat       -> nextHeartBeat(...)
TestRequest     -> nextTestRequest(...)
ResendRequest   -> nextResendRequest(...)
SequenceReset   -> nextSequenceReset(...)
Logout          -> nextLogout(...)
```

普通业务消息则进入 Application：

```
Session.next(Message)
-> verify(...)
-> Application.fromApp(...) / fromAdmin(...)
-> MessageCracker.crack(...)
-> onMessage(...)
```

核心原则：

```
Session 先判断“消息是否符合会话规则”；
Application 再判断“业务消息应该如何处理”。
```

2. 出站消息发送线

业务代码创建 Message

```
-> Session.sendToTarget(...)
-> Session.send(...)
-> sendRaw(...)
-> initializeHeader(...)
-> Application.toApp(...) / toAdmin(...)
-> MessageStore 持久化
-> Responder.send(...)
```

Session 会自动补充或覆盖会话字段：

```
BeginString(8)
SenderCompID(49)
TargetCompID(56)
MsgSeqNum(34)
SendingTime(52)
```

因此业务层通常不应自行维护 **MsgSeqNum(34)**。消息发送成功通常只表示消息已经排入异步网络发送队列，不代表对方已经完成业务接收；订单最终状态仍应以对方的 **ExecutionReport** 为准。

3. 定时器驱动线

定时器调用 Session.next()

- ├ 检查 Session 时间窗口
- ├ 发起或检查 Logon
- ├ 检查 Logout 超时
- ├ 检查 Heartbeat 超时
- ├ 发送 TestRequest
- └ 发送 Heartbeat

本方在 HeartBtInt 内没有发送任何消息

```
-> generateHeartbeat()
-> Heartbeat(35=0)
```

本方长时间没有收到对方任何消息

```
-> generateTestRequest()
-> TestRequest(35=1)
```

对方连 TestRequest 都没有响应

```
-> disconnect(...)
```

这里的两个 **next** 入口要区分：


```
next(Message)
= 收到 FIX 消息后处理

next()
= 定时器触发后主动维护 Session
```

4. 断线恢复与消息重传线

网络断开

- > Connector / Initiator 重建 TCP 连接
- > Session 重新发送或接收 Logon(35=A)
- > 比较双方 MsgSeqNum(34)
- > 发现序号缺口
- > ResendRequest(35=2)
- > MessageStore 重放历史消息

或

SequenceReset(35=4) / Gap Fill

- > 处理暂存消息
- > Session 恢复

当本方期望收到 34=52，却先收到 34=55 时：

收到 55

- > 判断序号过高
- > 将 55 放入 MessageQueue
- > 请求补发 52~54
- > 处理重传消息
- > 再处理队列中的 55

历史消息重发时通常带有：

```
PossDupFlag(43)=Y
OrigSendingTime(122)=原始发送时间
```

表示这是一条历史消息的重发，而不是新的业务请求。

四条主线如何共同完成一笔订单

业务创建 NewOrderSingle

- > 出站线：补 Header、分配序号、持久化并发送
- > 入站线：对手方接收、校验并进入 onMessage
- > 对手方返回 ExecutionReport

- > 定时线：空闲时维护 Logon、Heartbeat 和 TestRequest
- > 若网络断开，恢复线重新 Logon、比较序号并重传

四条主线对应的源码入口

主线	关键方法	主要职责
进站消息	<code>next(Message)</code> 、 <code>verify(...)</code> 、 <code>fromCallback(...)</code>	接收、校验和分发
出站消息	<code>send(...)</code> 、 <code>sendRaw(...)</code> 、 <code>persist(...)</code>	补 Header、分配序号、持久化和发送
定时器	<code>next()</code> 、 <code>generateLogon()</code> 、 <code>generateHeartbeat()</code> 、 <code>generateTestRequest()</code>	登录、心跳、探活和超时
恢复重传	<code>nextLogon(...)</code> 、 <code>nextResendRequest(...)</code> 、 <code>manageGapFill(...)</code> 、 <code>resendMessages(...)</code>	重连、补缺、重放和 Gap Fill

建议阅读源码时始终围绕四个问题：

- 消息能否进入？
- 消息如何发出？
- 没有业务消息时 Session 如何运行？
- 连接或序号出错后如何恢复？

5.1 Logon：FIX 会话建立的起点

在理解重连、序号恢复和消息重传之前，应该先理解 Logon，因为整个 FIX Session 的正式建立都从 35=A 开始。

```
Initiator
-> Session.next()
-> generateLogon()
-> initializeHeader(...)
-> 生成 35=A(Logon)
-> 写入 34=MsgSeqNum、49/56、98、108
-> toAdmin(...)
-> 发给对端

Acceptor
-> 收到 35=A
-> Session.nextLogon(...)
-> 检查 BeginString、Sender/Target、MsgSeqNum、会话状态
-> fromAdmin(...)
-> 校验通过
-> 回一个 35=A(Logon)
-> onLogon(sessionID)
```

Initiator

- > 收到对端 35=A
- > nextLogon(...)
- > 校验通过
- > onLogon(sessionID)
- > 会话正式进入 logged-on 状态

三个回调的职责可以压缩为：

toAdmin

- = 发出 Logon 前最后一次修改管理消息的机会

fromAdmin

- = 收到 Logon 后做认证和校验的入口

onLogon

- = 双方确认登录成功后的回调

5.2 Heartbeat：空闲时的会话保活机制

Heartbeat(35=0) 不能简单理解成“每隔 HeartBtInt 秒，对方向本方发一条心跳”。更准确的理解是：

在一个 FIX Session 中，双方都会根据 HeartBtInt 维护各自的会话活性。
如果某一方向在 HeartBtInt 时间内没有发送任何消息，
就会发送 35=0 Heartbeat，表示该方向仍然存活。

所以，HeartBtInt 的本质不是“强制固定周期发心跳”，而是：

空闲情况下的保活间隔

5.3 心跳不是只看对方发不发 35=0

FIX 的活性判断不是只靠单向心跳，而是双向维护：

Banzai -> Executor

这一方向维护自己的发送活性

Executor -> Banzai

这一方向也维护自己的发送活性

因此：

如果这段时间已经发送了其他正常消息
比如：

- 35=D NewOrderSingle
- 35=8 ExecutionReport
- 35=A Logon
- 35=5 Logout

那么这些消息本身就已经证明链路活着，
通常不一定还要额外发送 35=0。

一句话说就是：

Heartbeat 是“空闲时保活”，不是“无论是否有业务流量都机械定时发包”。

5.4 Heartbeat、TestRequest 与失联检测

可以把正常会话中的保活逻辑理解成下面这个流程：

正常有业务流量

- > 不一定需要额外 Heartbeat

长时间空闲

- > 发送 35=0 Heartbeat 保活

长时间没收到对方任何消息

- > 发送 35=1 TestRequest 探活

对方收到 TestRequest

- > 应回 35=0 Heartbeat

仍然没有响应

- > 判定连接异常，后续断开并重连

这里最容易混淆的点是：

35=0 Heartbeat

- = 空闲保活

35=1 TestRequest

- = 主动探测对方是否还活着

所以，Heartbeat 和 TestRequest 不是一回事，二者是配合关系。

5.5 如果 Logon 需要传用户名和密码

最常见的做法是：

客户端在 toAdmin(...) 里往 35=A 塞用户名和密码
 服务端在 fromAdmin(...) 里读取并校验
 校验失败则 RejectLogon
 校验通过才进入 onLogon(...)

流程图：

客户端

```
-> generateLogon()
-> toAdmin(message, sessionID)
    -> if 35=A
        set Username(553)
        set Password(554)
-> 发出 Logon
```

服务端

```
-> 收到 Logon
-> fromAdmin(message, sessionID)
    -> if 35=A
        读取 Username / Password
        做认证校验
        失败 -> RejectLogon
        成功 -> 继续
-> nextLogon(...)
-> 回 Logon
-> onLogon(sessionID)
```

最小示例：

```
@Override
public void toAdmin(Message message, SessionID sessionID) {
    try {
        String msgType = message.getHeader().getString(MsgType.FIELD);
        if (MsgType.LOGON.equals(msgType)) {
            message.setField(new Username("demo_user"));
            message.setField(new Password("demo_pass"));
        }
    } catch (FieldNotFound e) {
        throw new RuntimeException(e);
    }
}

@Override
public void fromAdmin(Message message, SessionID sessionID)
    throws FieldNotFound, IncorrectDataFormat, IncorrectTagValue, RejectLogon
{
}
```

```
String msgType = message.getHeader().getString(MsgType.FIELD);
if (MsgType.LOGON.equals(msgType)) {
    String username = message.getString(Username.FIELD);
    String password = message.getString>Password.FIELD);

    if (!"demo_user".equals(username) || !"demo_pass".equals(password)) {
        throw new RejectLogon("invalid username or password");
    }
}
}
```

对应 Logon 报文可以理解为：

```
8=FIX.4.4|9=...|35=A|34=1|49=CLIENT|56=SERVER|98=0|108=30|553=demo_user|554=demo_p
ass|10=...
```

注意：在 **FIX.4.2** 场景下，**Username(553)**、**Password(554)** 常常也会被使用，但是否能通过严格校验，取决于双方约定和 DataDictionary 配置。

5.6 Acceptor 动态 Session：Executor 如何按 Logon 创建不同会话

Executor 的动态 Session 机制适用于卖方网关、券商或交易所：服务端启动时不必预先为每个买方创建固定 Session；某个买方首次连接并发送 **Logon(35=A)** 后，Executor 才按模板创建该买方对应的真实 Session。

```
模板 = 未来某类客户应使用的配置
动态 Session = 某个真实客户首次 Logon 后实际创建的 Session 对象
```

5.6.1 模板配置：按 FIX 版本定义规则

executor_dynamic.cfg 为每个版本定义一个 **AcceptorTemplate=Y** 的模板：

```
[default]
ConnectionType=acceptor
SocketAcceptPort=9876
SenderCompID=*
TargetCompID=*

[session]
AcceptorTemplate=Y
BeginString=FIX.4.2
DataDictionary=FIX42.xml

[session]
AcceptorTemplate=Y
BeginString=FIX.4.4
DataDictionary=FIX44.xml
```

含义：

收到 FIX.4.2 客户的 Logon -> 使用 FIX42.xml 创建会话
收到 FIX.4.4 客户的 Logon -> 使用 FIX44.xml 创建会话

当前示例的 `SenderCompID=*`、`TargetCompID=*` 表示接受任意 CompID，仅适合学习。生产中必须结合明确的 CompID 规则、来源 IP 白名单、认证和 TLS 等限制接入方。

5.6.2 `configureDynamicSessions(...)` 启动时只注册“建会话工厂”

源码：`quickfixj-`

`examples/executor/src/main/java/quickfix/examples/executor/Executor.java`。

```
Executor 启动
|
▼
遍历 SessionSettings 中全部 [session]
|
▼
筛选 AcceptorTemplate=Y 的模板
|
▼
按 SocketAcceptAddress + SocketAcceptPort 分组 TemplateMapping
|
▼
为每个监听地址注册 DynamicAcceptorSessionProvider
|
▼
SocketAcceptor 开始监听
```

`TemplateMapping(sessionID, sessionID)` 的两个参数在当前示例中看起来相同，但语义不同：

```
pattern      = 哪些真实 SessionID 可以匹配模板
templateID = 匹配成功后应复制哪段 Session 配置
```

因此，`configureDynamicSessions(...)` 本身不会创建 **FUND_A**、**BANZAI** 等真实客户 **Session**；它只为端口安装后续动态创建 Session 的 Provider。

5.6.3 首次 Logon 时的动态创建流程

```

买方 FUND_A 连接 Executor:9876
|
| Logon(35=A)
| 8=FIX.4.2, 49=FUND_A, 56=EXEC
|
▼
AcceptorIoHandler
|
| 按 Acceptor 本地视角反转身份
|
▼
真实 SessionID: FIX.4.2:EXEC->FUND_A
|
|
▼
DynamicAcceptorSessionProvider.getSession(sessionID)
|
| 已存在该 Session
|   └─ 直接复用已有的 Session、Store 和序号状态
|
| 不存在
|
|   ▼
|   匹配模板: FIX.4.2:*->*
|   |
|   |   ▼
|   |   复制 [default] + FIX.4.2 模板配置
|   |   |
|   |   | DataDictionary=FIX42.xml
|   |   | HeartBtInt、FileStorePath、日志等通用参数
|   |   | └─ Application、MessageStoreFactory、MessageFactory
|   |   |
|   |   ▼
|   |   用 Logon 中真实身份覆盖 BeginString / SenderCompID / TargetCompID
|   |   |
|   |   ▼
|   |   DefaultSessionFactory.create(...)
|   |   |
|   |   ▼
|   |   sessionConnector.addDynamicSession(session)
|   |   |

```


▼
将 Session 绑定当前 TCP 连接并继续完成 Logon

同一端口因此可同时维护多个独立 Session:

```
FIX.4.2:EXEC->FUND_A  -> FIX42.xml、独立 MsgSeqNum、独立 Store
FIX.4.4:EXEC->FUND_B  -> FIX44.xml、独立 MsgSeqNum、独立 Store
FIX.4.1:EXEC->FUND_C  -> FIX41.xml、独立 MsgSeqNum、独立 Store
```

5.6.4 与多 Session 和高并发的关系

动态 Session

= 按需管理大量潜在接入方的生命周期。

多 Session

= 多个真实会话之间的连接、序号、心跳、Store 和恢复状态彼此隔离。

高并发

= 多条 Session 可并行通信；同一条 Session 内仍必须严格保持消息顺序。

Dynamic Acceptor Session 解决“未知或大量客户如何按需接入”；它不是单条 Session 的吞吐优化，也不等同于系统级高可用。

5.7 为什么不只讲“断线重连”

仅有 Socket 重连不足以保证 FIX 会话正确：

连接恢复了

≠

双方确认没有丢消息、没有错序、没有重复处理消息

FIX Session 的关键问题是：

网络中断后，双方如何恢复到一致的消息序列状态？

因此，第三条主线的准确名称应为：

Session 可靠性：心跳、断线、重连、序号恢复与消息重传

5.8 正常会话中的可靠性机制

每个逻辑 Session

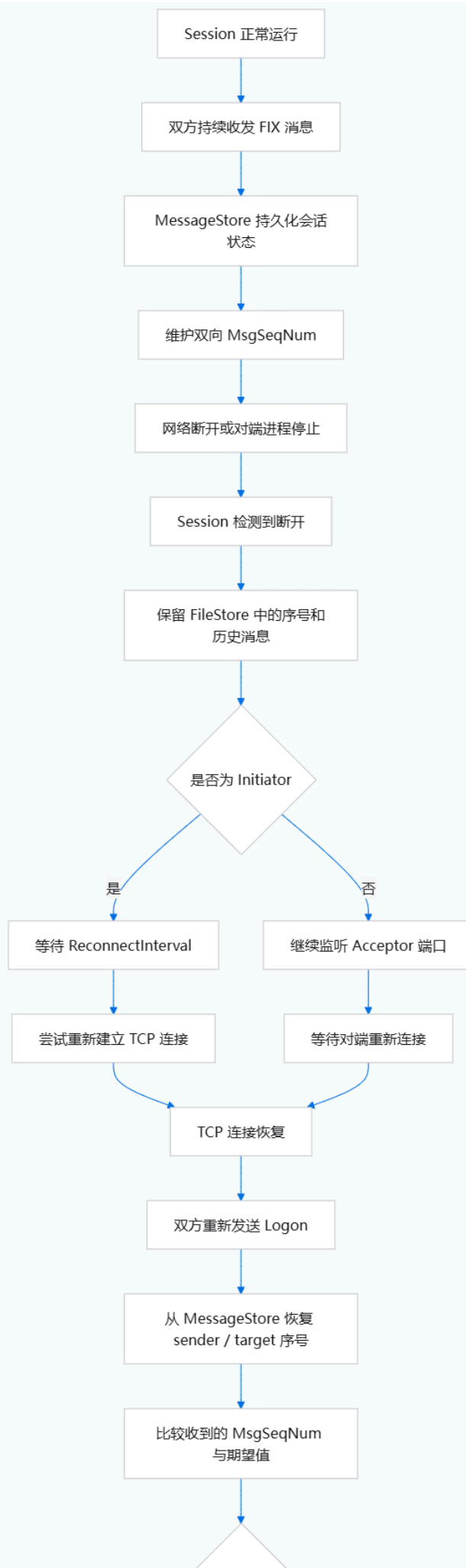
- | 每个方向独立维护 MsgSeqNum(34)
- | MessageStore 持久化消息和序号
- | Heartbeat(35=0) 保持活性
- | TestRequest(35=1) 探测疑似失联的对端

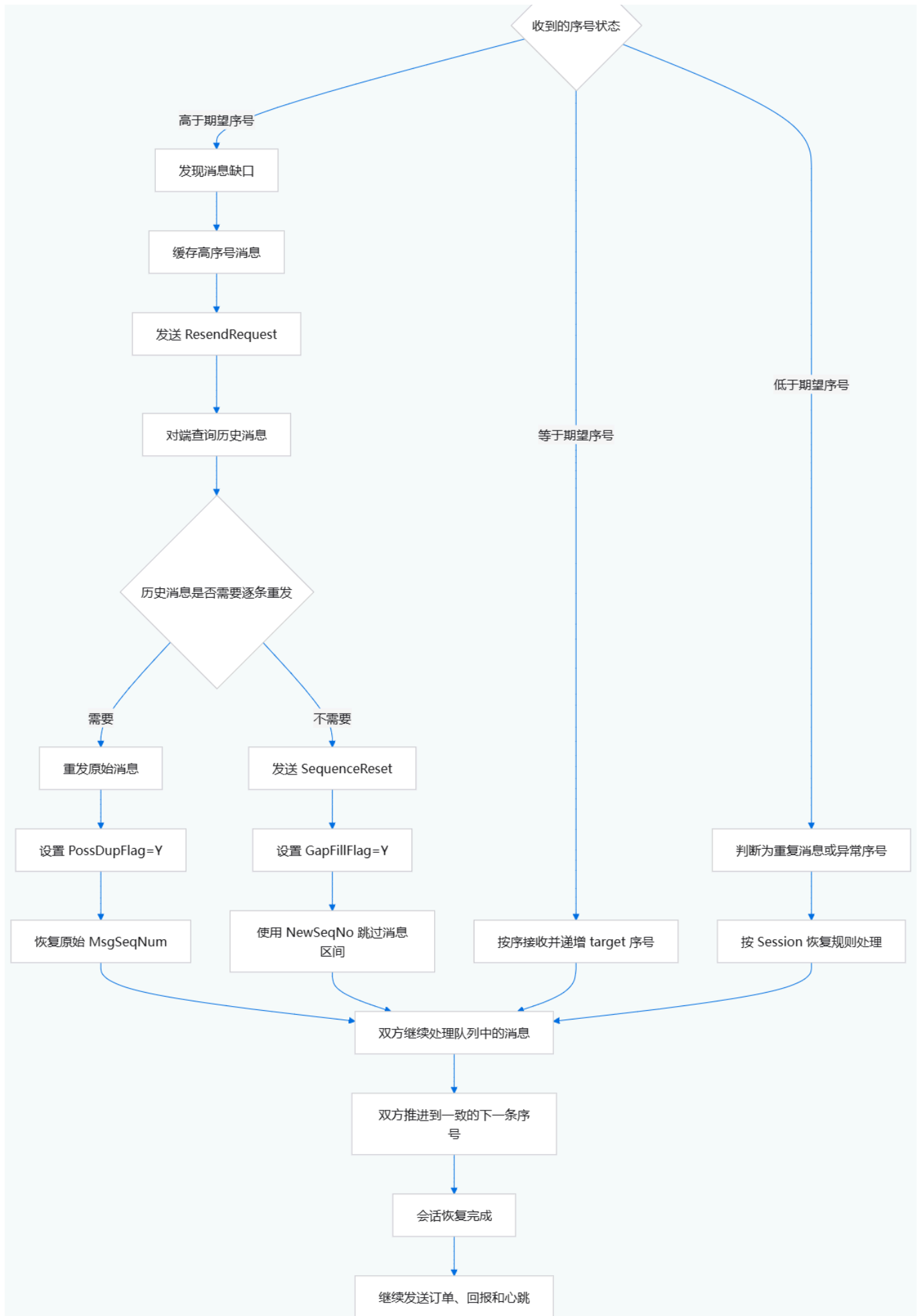
5.9 断线恢复流程





双方序号重新一致，Session 恢复





5.10 关键概念表

概念	含义	要解决的问题
MsgSeqNum(34)	单方向消息序列号	如何发现乱序或缺失消息？

概念	含义	要解决的问题
MessageStore	保存消息和收发序号	重连后如何知道历史状态？
FileStore	基于文件的 Store 实现	学习和本地演示中如何持久化状态？
Heartbeat(35=0)	定期心跳	空闲时如何确认连接存活？
TestRequest(35=1)	主动健康检查	怀疑对端失联时如何探测？
ResendRequest(35=2)	请求缺失序号范围	发现缺号后如何补消息？
PossDupFlag(43)	重发标识	收到重发业务消息时如何识别？
SequenceReset(35=4)	序号重置或 Gap Fill	不逐条重发时如何推进序号？
ResetSeqNumFlag(141)	Logon 时重置序号标志	在双方明确同意时如何从头开始？

5.11 应用层仍需具备幂等能力

即使 FIX 有序号和 PossDupFlag(43)，业务系统也不能假设“同一业务结果绝不会再出现”。

应用层通常需要根据：

- ClOrdID
- OrigClOrdID
- ExecID
- OrderID

识别重复请求、重复执行回报或重发消息，避免重复记账、重复成交、重复更新状态。

5.12 Session 可靠性与系统级高可用的边界

QuickFIX/J 可以提供：

- 会话级自动重连
- 收发序号管理
- 心跳与失联探测
- 消息重传
- 本地消息和序号持久化

QuickFIX/J 不会自动提供：

- 多实例双活
- 主备切换编排
- 分布式一致性
- 多机共享的 Session 状态
- 容灾与网络冗余
- 监控、告警、容量治理

因此：

QuickFIX/J 的 Session 可靠性
≠
完整交易系统的高可用架构

完整生产方案通常还需要：

进程监控 / 容器编排
主备或集群部署
可恢复或共享的持久化 Store
网络冗余
监控、告警、审计
业务幂等与重复回报处理

5.13 对外介绍时的关键结论

QuickFIX/J 不只是“把消息发到 Socket 上”。它通过 Session、序号、Store、心跳和重传机制，使断线后的通信可以恢复到双方可验证的一致状态。

6. 奇点和 QuickFIX 的异同分析

6.1 为什么这组对比有价值

如果把 QuickFIX/J 只看成“一个收发 FIX 报文的库”，就很难把它和真实交易系统联系起来。

而奇点这类策略交易系统通常已经把：

策略参数/状态
行情接入
交易接入
回调处理
交易决策

组织成了完整应用结构。

因此，把奇点和 QuickFIX/J 放在一起看，最有价值的地方不是比较“谁功能更多”，而是弄清楚：

奇点：更像一个已经组装好的策略交易系统
QuickFIX/J：更像一个基于 FIX 标准的交易通信与会话基础设施

6.2 两者的整体定位差异

奇点交易系统

= 策略系统 + 行情 API + 交易 API + 回调处理 + 交易决策

QuickFIX/J

= FIX 协议引擎 + Session 会话管理 + Message 模型 + DataDictionary + Application 回调边界 + 示例业务应用

QuickFIX/J 本身不是完整策略系统，Banzai、Executor、OrderMatch 只是帮助理解 FIX 消息收发、会话恢复和业务接入边界的示例。

6.3 架构角色对应关系

奇点交易系统	QuickFIX/J 学习系统
策略参数 / 状态 xmdapi + MdSpi traderapi + TraderSpi 理 交易前置 策略与交易决策 行扩展	Banzai 本地订单状态、后续可扩展的策略状态 FIX 行情 Session + fromApp 中的行情处理 Session.sendToTarget + fromApp 中的回报处 Executor / OrderMatch 作为对端 Acceptor QuickFIX/J examples 默认不完整提供，需要自

更准确地说：

MdSpi
≈ QuickFIX/J Application.fromApp(...) 中的行情消息处理

TraderSpi / SimpleTraderSpi
≈ QuickFIX/J Application.fromApp(...) 中的订单回报处理

ReqOrderInsert / 报单请求
≈ NewOrderSingle + Session.sendToTarget(...)

6.4 完全成交流程对照

前面的角色对应关系说明了奇点交易系统与 QuickFIX/J 学习系统中各个组件的大致位置。下面用两张完全成交流程图，把两者从报单到最终成交回报的过程放在一起对照。

6.4.1 奇点系统的订单完全成交流程

● 场景三：报单被交易所接收并成交(全部成交)

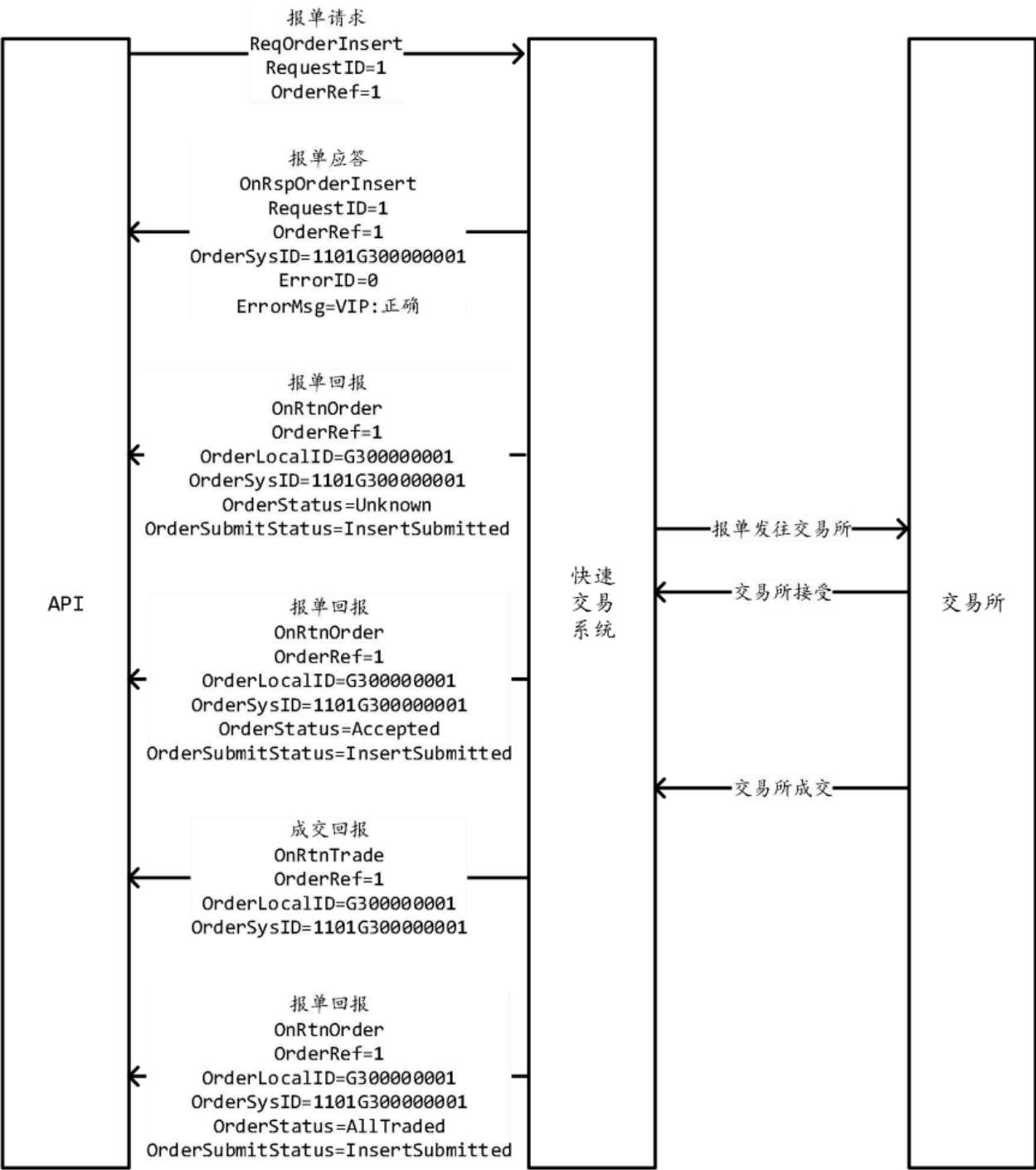


图 6-1：奇点系统从报单请求到完全成交的完整回调链路。

这张图的主线是：

```
ReqOrderInsert
-> OnRspOrderInsert
-> OnRtnOrder (已接收)
-> OnRtnOrder (已接受)
```

```
-> OnRtnTrade  
-> OnRtnOrder (AllTraded)
```

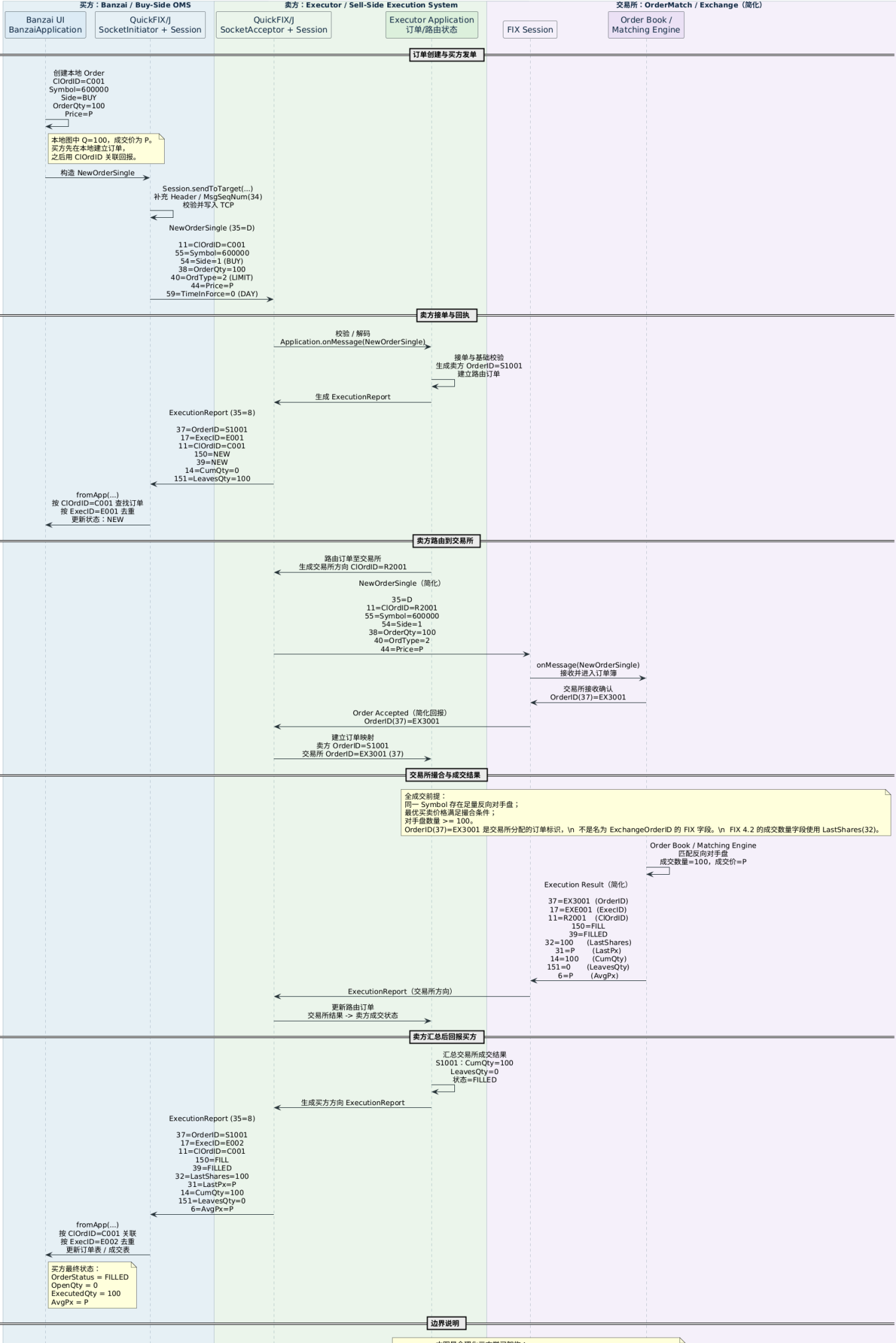
重点关注两个维度：

```
OrderStatus  
= 订单当前处于什么状态  
  
OrderSubmitStatus  
= 报单或撤单请求处于什么提交阶段
```

也就是说，奇点通过 API 请求和 SPI 回调表达订单生命周期，完全成交通常由成交回调 `OnRtnTrade` 与最终的 `OnRtnOrder` 共同体现。

6.4.2 QuickFIX/J 的订单完全成交流程

QuickFIX/J 三方订单完全成交安全流程



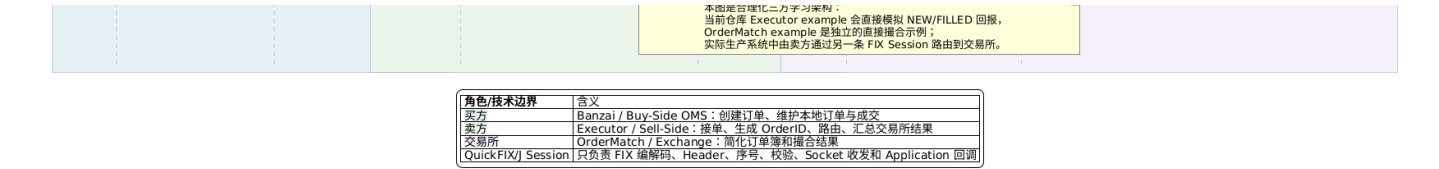


图 6-2：基于 QuickFIX/J 角色抽象的买方、卖方和交易所三方完全成交链路。

这张图的主线是：

买方 Banzai

-> NewOrderSingle(35=D)

-> 卖方 Executor 接单

-> ExecutionReport(35=8, OrdStatus=NEW)

-> 卖方路由到交易所

-> 交易所返回成交结果

-> 卖方汇总成交结果

-> ExecutionReport(35=8, OrdStatus=FILLED)

-> 买方更新订单和成交

QuickFIX/J 图中需要重点观察：

C1OrdID(11)

= 当前发单方生成的客户订单号

OrderID(37)

= 当前接收方或执行场所分配的订单号

ExecID(17)

= 每一次执行回报事件的唯一标识

在三方链路中，不同 Session 可以拥有不同的订单标识：

买方发给卖方：C1OrdID=C001

卖方自己的订单：OrderID=S1001

卖方发给交易所：C1OrdID=R2001

交易所返回：OrderID(37)=EX3001

卖方通过内部映射把交易所订单关联回自己的订单，再使用买方原始的 C1OrdID=C001 向买方发送最终成交回报。

说明：QuickFIX/J 图展示的是买方、卖方和交易所之间的合理化三方业务链路。当前仓库中的 Executor example 会直接模拟生成 NEW/FILLED 回报，OrderMatch example 则是独立的撮合示例；两者尚未在代码中组成完整的 Banzai → Executor → OrderMatch 三方运行链路。

6.4.3 两张流程图的核心对应关系

交易流程	奇点系统	QuickFIX/J 学习系统
发起报单	ReqOrderInsert	NewOrderSingle(35=D) + Session.sendToTarget(...)
接单/报单 响应	OnRspOrderInsert	ExecutionReport(35=8, ExecType=NEW)
委托状态回 报	OnRtnOrder	ExecutionReport(35=8, OrdStatus(39))
成交回报	OnRtnTrade	ExecutionReport(35=8, ExecType(150)=FILL/TRADE)
完全成交状 态	OrderStatus=AllTraded	OrdStatus(39)=FILLED、LeavesQty(151)=0
本地订单关 联	OrderRef、OrderLocalID、 OrderSysID	ClOrdID(11)、OrderID(37)、ExecID(17)
回调处理入 口	TraderSpi	Application.fromApp(...) / MessageCracker

两张图最重要的共同点是：

发出订单请求

-> 收到接单或接受反馈

-> 收到成交事件

-> 更新本地订单状态

最重要的差异是：

奇点

= 通过专用 API 和结构化回调表达业务状态

QuickFIX/J

= 通过 FIX Message、Tag、Session 和 Application 回调表达业务状态

6.5 共同点：两者都在做“事件驱动交易”

行情输入

-> 更新本地状态

-> 触发策略或交易员动作

-> 生成订单请求

-> 发送给交易对端

-> 接收订单状态和成交回报

-> 更新订单、持仓、资金状态

两者在系统形态上有 5 个共同点：

1. 都是事件驱动
2. 都采用异步请求 + 异步回调
3. 都需要维护本地订单/持仓/行情状态
4. 都有行情链和交易链两条主线
5. 都必须处理断线、重连与恢复

6.6 不同点一：协议和接口形态不同

奇点

= 专用交易 API / SDK

= 开发者直接调用行情 API、交易 API 和 SPI 回调

QuickFIX/J

= FIX 标准协议引擎

= 开发者处理的是 Session、Message、Tag、DataDictionary 和 Application 回调

这意味着：

奇点偏“API 调用模型”

QuickFIX/J 偏“协议消息模型”

例如：

奇点：

SubscribeMarketData(...)

ReqOrderInsert(...)

OnRtnOrder(...)

OnRtnTrade(...)

QuickFIX/J：

MarketDataRequest

NewOrderSingle

ExecutionReport

OrderCancelRequest

fromApp(...)

6.7 不同点二：回调边界不同

奇点通常会天然拆成：

MdSpi

└ 处理行情事件

TraderSpi

- └ 处理登录、报单、委托状态、成交回报

QuickFIX/J 默认只有一个统一边界：

Application

- └ onCreate / onLogon / onLogout
- └ toAdmin / fromAdmin
- └ toApp / fromApp

所以 QuickFIX/J 的使用者通常还需要在 **Application** 之后自行再分层：

Application

- └ MarketDataHandler
- └ OrderHandler
- └ ExecutionReportHandler
- └ StrategyEngine

也就是说：

奇点：框架天然帮你拆好了行情和交易接口

QuickFIX/J：先给你统一的 FIX 边界，业务分层由你自己决定

6.8 不同点三：QuickFIX/J 对 Session 可靠性暴露得更清楚

奇点类 API 往往把连接、登录、重连细节大部分封装在 SDK 中；开发者主要感知：

OnFrontConnected

登录响应

订阅成功

报单回报

成交回报

QuickFIX/J 则把 FIX Session 的关键机制显式暴露出来：

SessionID

Logon / Logout

Heartbeat / TestRequest

MsgSeqNum(34)

ResendRequest(35=2)

SequenceReset(35=4)


```
MessageStore / FileStore  
ReconnectInterval
```

所以：

奇点更强调“如何调用交易服务”
QuickFIX/J 更强调“如何维护标准 FIX 会话并保证消息可靠传输”

6.9 不同点四：消息模型不同

奇点类系统通常以结构体、请求对象和回调参数为主：

```
InputOrderField  
RspOrderInsert  
RtnOrder  
RtnTrade
```

QuickFIX/J 则以 FIX 协议消息模型为主：

```
FieldMap  
Message  
Header / Body / Trailer  
DataDictionary  
NewOrderSingle  
ExecutionReport
```

二者本质区别可以压缩为：

奇点：结构体 / API 驱动
QuickFIX/J：FIX Message / DataDictionary 驱动

6.10 最重要的边界结论

奇点交易系统
= 更接近完整策略交易应用

QuickFIX/J
= 更接近交易通信、协议表达、会话恢复的底层基础设施

因此，Banzai、Executor、OrderMatch 更适合帮助回答：

- 一笔订单如何被 FIX 表达并发送？
- 对端如何解析、分派并生成回报？
- 断线后如何通过 MsgSeqNum、Store、ResendRequest 恢复会话？

而如果要做成“奇点那样”的系统，还需要在 QuickFIX/J 之上继续补：

策略状态管理
行情缓存
风控前置
订单路由
持仓与资金管理
交易后处理
监控与运维能力

6.11 一句话记忆

奇点告诉你：如何围绕行情和交易 API 组织一个策略交易系统。
QuickFIX/J 告诉你：如何用 FIX 标准把行情、订单、回报和会话恢复可靠地表达与传输。

7. 后续重点：QuickFIX/J 高并发交易处理

高并发是 QuickFIX/J 从示例走向真实交易系统时必须重点理解的主题。其最重要的底层基础之一就是 **多 Session 通信**：买方系统可同时维护多个行情源、多个券商、多个交易场所或 Drop Copy 的独立 FIX 会话。

这里的“高并发”不是让同一个 FIX Session 内的消息无序并行，而是建立下面的并发模型：

多个 Session 并行
+
同一 Session 内严格有序
+
业务处理与网络 IO 解耦
+
消息与序号持久化
+
多实例 / 多进程水平扩展

7.1 买方多 Session 通信：高并发的会话基础

7.1.1 核心原则

一个 QuickFIX/J 应用进程
└ 一个 SocketInitiator

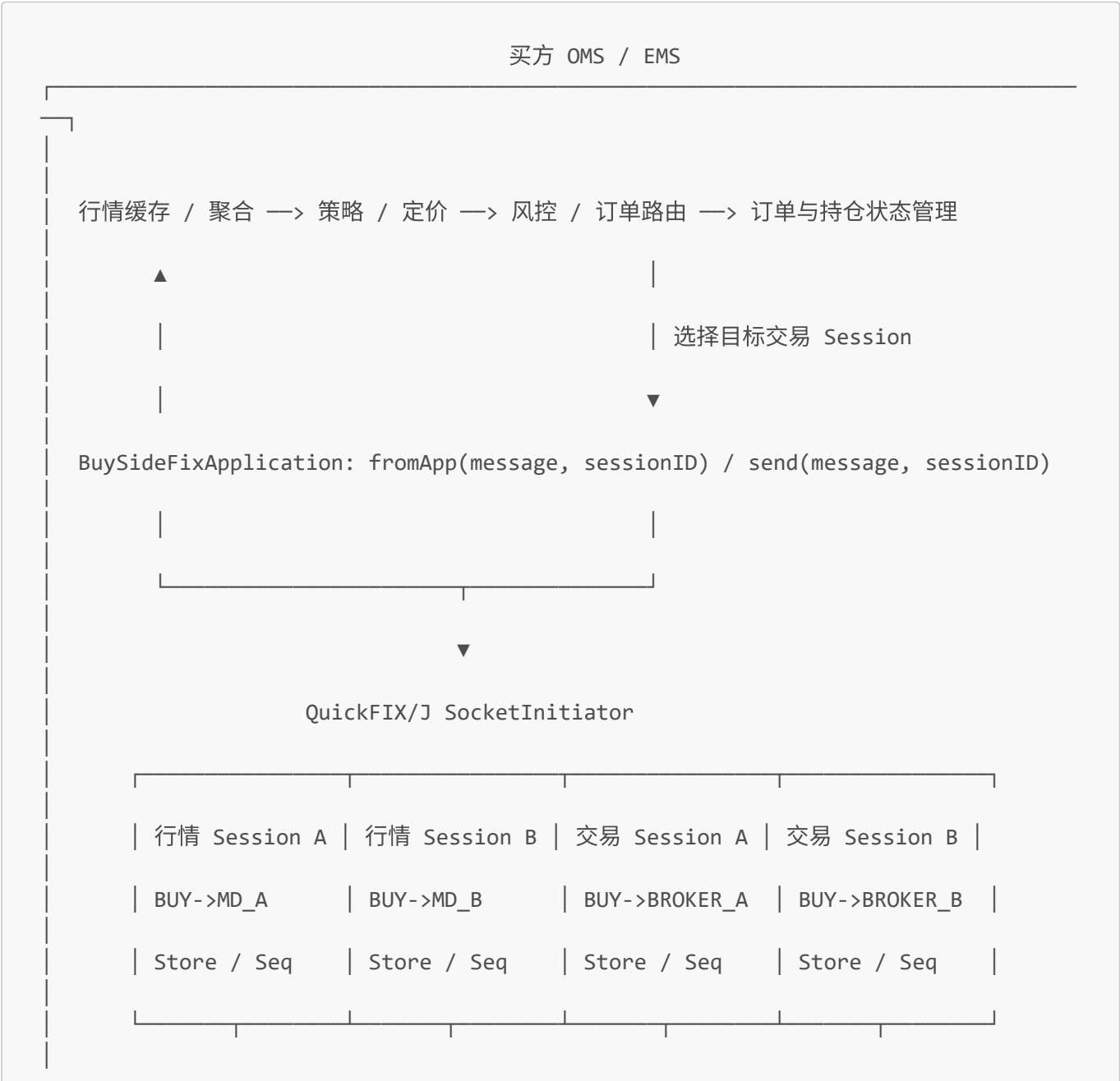
└ 根据 SessionSettings 中多个 [session] 创建并管理多条逻辑 Session

- 每条 Session 都独立拥有：
- SessionID：逻辑身份与业务路由键
 - TCP 连接及 Logon / Logout 状态
 - Sender / Target MsgSeqNum(34)
 - Heartbeat、TestRequest、重连与重传状态
 - MessageStore、日志和恢复记录

因此，多 Session 不是“把不同对手方的消息混到一条连接中”，而是：

QuickFIX/J 负责隔离并维护多条 FIX 逻辑会话；
买方 OMS / EMS 根据 SessionID 把消息路由到正确的行情、订单或回报处理链路。

7.1.2 买方 OMS / EMS 多 Session 总图





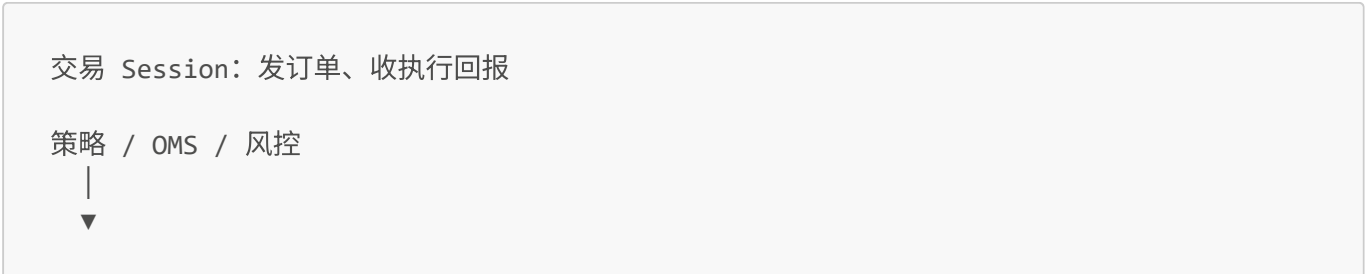
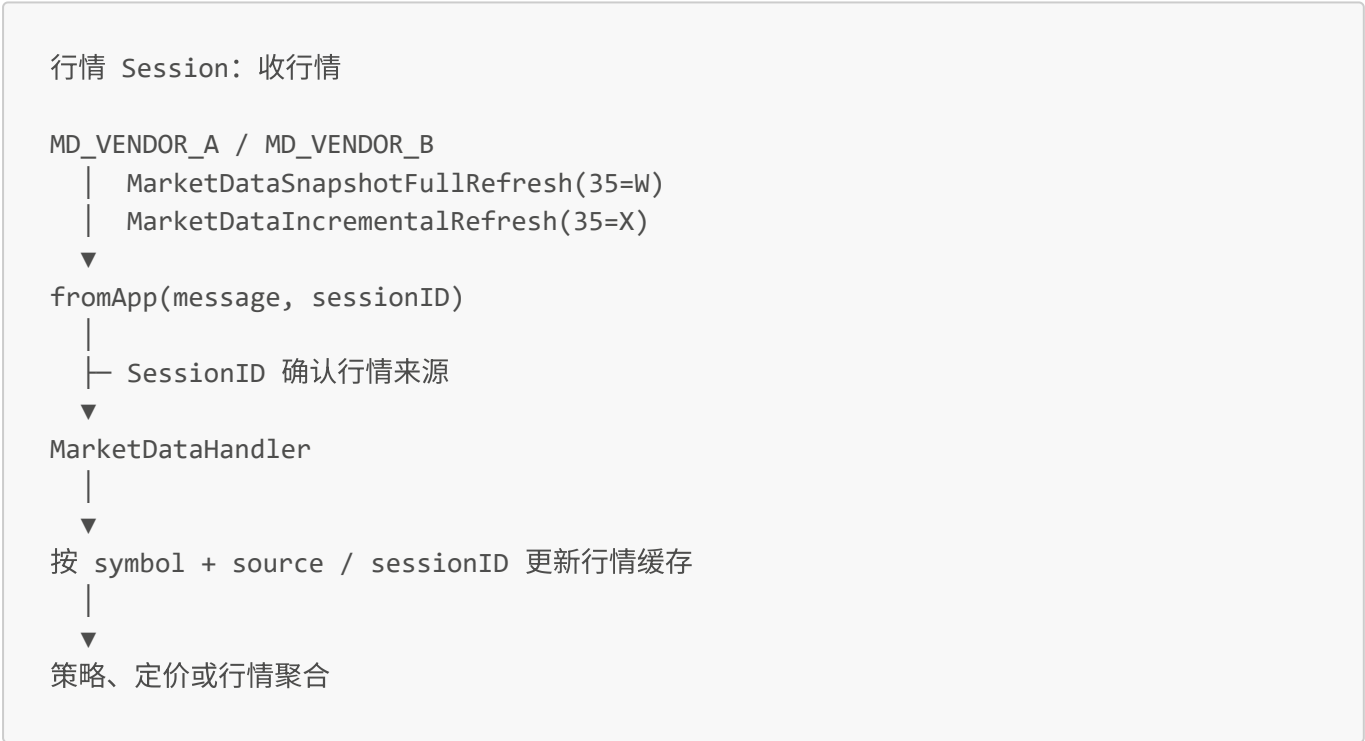
示例 SessionID:

FIX.4.4:BUY_SIDE->MD_VENDOR_A	行情源 A
FIX.4.4:BUY_SIDE->MD_VENDOR_B	行情源 B
FIX.4.4:BUY_SIDE->BROKER_A	券商 / 场所 A
FIX.4.4:BUY_SIDE->BROKER_B	券商 / 场所 B

同一个 **Application** 实例可以处理全部 Session 的回调，但它们绝不共享序号和会话状态：

BUY_SIDE->MD_VENDOR_A 的 MsgSeqNum
≠ BUY_SIDE->MD_VENDOR_B 的 MsgSeqNum
≠ BUY_SIDE->BROKER_A 的 MsgSeqNum
≠ BUY_SIDE->BROKER_B 的 MsgSeqNum

7.1.3 行情与交易的两类通信流程



```

OrderRouter 根据流动性、费用、可用性等规则选择 Broker A 或 Broker B
    |
    ▼
Session.sendToTarget(NewOrderSingle, targetSessionID)
    |
    ▼
Broker / Venue
    |
    ▼
ExecutionReport(35=8)
    |
    ▼
fromApp(executionReport, sessionID)
    |
    ├── SessionID 确认回报所属券商 / 场所
    └── ClOrdID / OrderID / ExecID 关联本地订单并更新状态
  
```

消息类型回答“这是什么消息”；**SessionID** 回答“它来自哪里，或应该发往哪里”。两者必须同时参与业务路由。

7.1.4 隔离、故障与高并发的关系

```

MD_VENDOR_A 断线
    |
    ▼
只影响 FIX.4.4:BUY_SIDE->MD_VENDOR_A
    ├── 该 Session 独立按 ReconnectInterval 重连
    ├── 该 Session 独立根据自己的 Store 恢复 MsgSeqNum
    ├── 该 Session 独立处理 ResendRequest / SequenceReset
    └── 不应影响 MD_VENDOR_B、BROKER_A、BROKER_B 的通信
  
```

这提供了高并发系统所需的基础隔离：

连接隔离 + 会话状态隔离 + 对手方隔离 + 业务用途隔离

但要准确区分边界：多 Session 支持 **并行通信和故障隔离**，并不天然等于完整高可用。若全部 Session 位于同一个 JVM，进程崩溃仍会影响全部会话；主备切换、跨机 Store 恢复、监控告警、限流和容量治理仍需要系统层设计。

7.1.5 多 Session 的实现规则

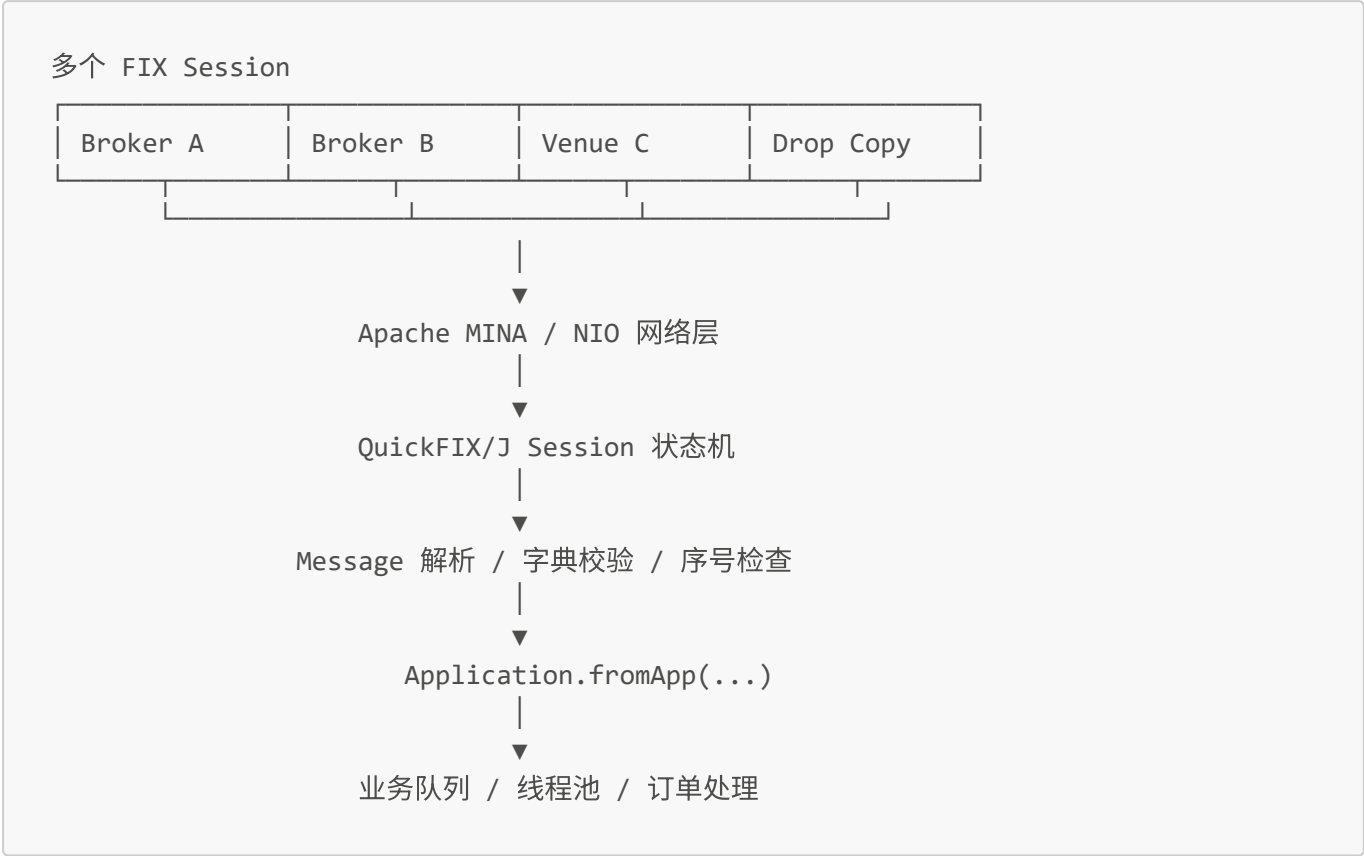
1. 将 SessionID 视为业务路由键，而非仅用于日志展示。
2. 每条 Session 的 MsgSeqNum、Store、重传和心跳状态完全独立。
3. 行情 Session 与交易 Session 在业务层分域；不能混淆其消息和可用性。
4. 发送消息时显式指定目标：Session.sendToTarget(message, sessionID)。

5. 接收消息时同时按“消息类型 + SessionID”分派。

6. 不同 Session 可以并行；同一 Session 的 FIX 协议消息必须保持顺序。

7. 买方业务层仍要统一维护跨 Session 的行情聚合、订单路由、持仓、风控和审计。

7.2 QuickFIX/J 的并发架构



Session A 与 Session B 可以并行推进。

但同一个 Session 内的消息通常必须保持顺序：
34=100 -> 34=101 -> 34=102

不能为了追求并发而破坏：

- MsgSeqNum(34)

- Logon / Logout 状态

- 订单状态顺序

- ResendRequest / SequenceReset 恢复逻辑

7.2 QuickFIX/J 支持高并发的关键机制

机制	作用	学习重点
Apache MINA / NIO	用异步网络模型管理多个 Socket 连接	理解网络 IO 与业务线程的边界
多 Session	不同券商、交易所、行情和 Drop Copy 会话可以并行	理解 SessionID 和 Session 隔离

机制	作用	学习重点
Session 状态机	在每个 Session 内保证消息顺序与协议状态	理解 <code>MsgSeqNum</code> 、Logon、Heartbeat、重传
MessageCracker	将通用消息分派到具体业务处理器	理解协议解析后的业务入口
MessageStore	保存序号和历史消息	理解高并发下的恢复、一致性和 IO 成本
业务队列/线程池	将耗时业务从 FIX 接收路径中隔离	避免阻塞 IO 或 Session 处理线程

7.3 业务层的正确并发边界

不要在 `fromApp(...)` 或 `onMessage(...)` 中直接执行长时间阻塞操作：

不推荐：

FIX 接收线程

- > 远程 RPC
- > 慢数据库查询
- > 大量策略计算
- > 长时间阻塞

更合理的结构是：

FIX 接收路径

- > 快速解析和校验
- > 放入有界业务队列
- > 立即返回

业务 Worker

- > 风控
- > 策略计算
- > 订单状态更新
- > 数据库或外部服务

为了同时保证顺序和并发，建议按业务键分区：

不同 Session：可以并行

不同订单：可以并行

同一 Session 的协议消息：保持有序

同一订单的状态变化：保持有序

常见分区键包括：

```
SessionID  
ClOrdID  
OrderID  
账户 / 组合 ID
```

7.4 高并发并不等于同一 Session 无序并行

一个订单的状态必须保持：
New -> PartiallyFilled -> Filled

如果把同一订单的回报交给多个无序线程，可能出现：

- Filled 先于 New 被处理
- 撤单先于订单创建被处理
- 旧回报覆盖新状态
- 重发消息导致重复记账

因此 QuickFIX/J 的正确思路是：

```
Session 之间并行  
Session 内部有序  
业务订单按键分区  
状态更新保持幂等
```

7.5 高并发与会话恢复的关系

高并发场景下，消息量越大，断线期间形成的序号缺口和待恢复消息可能越多。因此并发设计必须和 Session 可靠性一起考虑：

```
高并发收发  
-> MessageStore 持久化消息和序号  
-> 网络或进程故障  
-> 重连并重新 Logon  
-> 发现 MsgSeqNum 缺口  
-> ResendRequest  
-> 历史消息重发或 SequenceReset / GapFill  
-> 恢复到一致的 Session 状态
```

这说明：

```
并发性能解决“消息能否及时流动”  
Session 可靠性解决“消息是否有序、可恢复、不重复处理”
```


7.6 QuickFIX/J 高并发的边界

QuickFIX/J 可以提供：

- 多 Session 并行通信
- NIO 网络 IO
- FIX 消息解析与校验
- Session 内序号管理
- 心跳、重连和消息重传
- 本地消息与序号持久化

QuickFIX/J 不会自动提供：

- 高性能策略计算
- 业务风控
- 订单簿和撮合优化
- 分布式状态一致性
- 多实例双活和主备切换
- 业务级限流、背压和容量治理

所以生产级高并发系统通常需要在 QuickFIX/J 之上继续建设：

- QuickFIX/J
- + 业务线程池与有界队列
 - + 按 SessionID / ClOrdID 的消息分区
 - + 风控与订单路由
 - + 高性能订单、持仓和资金状态管理
 - + Store 性能评估与恢复策略
 - + 监控、告警、限流和背压
 - + 主备 / 集群 / 容灾部署

7.7 与奇点交易系统的对应关系

奇点	QuickFIX/J 扩展架构
xmdapi + MdSpi	FIX MarketData Session + MarketDataHandler
traderapi + TraderSpi	FIX Trading Session + ExecutionReportHandler
策略参数 / 状态	StrategyState
策略与交易决策	StrategyEngine
	PreTradeRiskEngine
	OrderRouter
	PositionManager

这也是后续实战学习的重点：

不仅要理解“QuickFIX/J 如何收发一条消息”，
还要理解“多个 Session、高吞吐消息流和业务线程池如何协同工作”。

7.8 后续学习与压测建议

- 第一步：观察多个 Session 是否可以独立运行

第二步：观察 fromApp 中阻塞业务对心跳和回报的影响

第三步：增加业务队列和 Worker，验证 Session 内顺序

第四步：模拟高频订单与回报，观察队列长度和延迟

第五步：压测 FileStore / MessageStore 的写入成本

第六步：模拟断线，观察高消息量下的 ResendRequest 和恢复耗时

记忆结论：QuickFIX/J 的并发模型是“多个 Session 并行、同一 Session 有序、业务处理异步化”。它提供交易通信的并发基础，但完整的高并发交易能力仍需要业务分区、线程模型、状态管理、存储优化和部署架构共同完成。

8. 推荐的分享结构

适合 30~45 分钟技术分享的结构：

第 0 部分：QuickFIX/J 的定位与边界	3~5 分钟
第 1 部分：一笔 NewOrderSingle 的端到端业务流程	10~15 分钟
第 2 部分：领域对象、FIX Message、原始报文的转换	10~15 分钟
第 3 部分：Session 可靠性与恢复机制	10~15 分钟
第 4 部分：奇点和 QuickFIX 的异同分析	5~8 分钟
第 5 部分：高并发交易处理的并发模型与实战边界	8~12 分钟
第 6 部分：总结与源码、实验入口	3~5 分钟

8.1 每部分建议包含的内容

部分	架构图	示例	建议源码入口	可演示实验
定位与边界	完整交易系统与 QuickFIX/J 的层次图	Banzai、OrderMatch、Executor 的角色	<code>quickfix.Application</code>	展示 Initiator / Acceptor 配置
下单流程	Banzai → OrderMatch → ExecutionReport	<code>35=D</code> 、 <code>35=8</code>	<code>BanzaiApplication.send42</code> 、 <code>OrderMatch.onMessage</code>	Banzai 登录、下单、收到回报

部分	架构图	示例	建议源码入口	可演示实验
消息转换	Order → Message → FIX 文本 → Message → Order	Header / Body / Trailer	Message、FieldMap、 DataDictionary、 MessageCracker	打印一条 真实 FIX 报文
Session 可靠性	Disconnect → Logon → Resend → Recover	35=A、35=2、35=4	Session.next()、 MessageStore	停止并重 启一端， 观察重连 与序号
奇点 vs QuickFIX	策略系统与 FIX 引擎对照图	xmdapi/MdSpi vs Application/fromApp	Application、Session、 DataDictionary	对照一条 行情链和 一条交易 链
高并发 处理	多 Session 并 行、Session 内有 序、业务异步化	NIO、线程池、队列、 Store	Session、MINA、 MessageStore	多 Session 和高频消 息压测

9. 最终收束图



10. 一句话版本

用一笔订单说明“业务怎么接入”，
用消息转换说明“协议怎么落地”，

用 Session 恢复说明“通信为什么可靠”。

最终结论：

以“下单业务链路、消息转换机制、Session 可靠性”作为三条主线是一个高质量的 QuickFIX/J 介绍框架。补上 QuickFIX/J 的边界，并把“断线重连”扩展为“序号、存储与重传驱动的会话恢复”，这套材料就能从学习笔记提升为完整的架构介绍。